

Lab 10 Correlated Subquery and Division

United International College
COMP3013 DBMS

Outline

- Division
- Correlated subquery

Division in Set Operations

In set operations, we have

- Union, which is basically addition
- Set difference, which is basically subtraction
- Cartesian product, denoted by $\times$, which is basically multiplication

What about division?

- Division, denoted by $\div$, is like the inverse of Cartesian product.
- Division operator is used for queries involving “all”.

Division Example

- Query: “Retrieve (all) course names that is/are taught by **all** programmes.”
 - The first “all” in parenthesis doesn’t count; every query requires that “all”
 - The second “**all**” suggests division should be used.

<i>catalogue</i>		<i>programme</i>	<i>catalogue ÷ programme</i>
c_name	p_name	p_name	c_name
Database	CST	CST	Database
OS	CST	DS	
Database	DS		
Data Analytics	DS		

- The solution to the query is *catalogue ÷ programme*
- Database is taught by all programmes, i.e. both CST and DS.
 - Note: here we are only interested in programmes in the DCS department.

Division Example

Verifying “division is like the **inverse** of Cartesian product”

$$(catalogue \div programme) \times programme = \{(Database, CST), (Database, DS)\}$$

- Both of these tuples are in *catalogue*
- So $(catalogue \div programme) \times programme \subseteq catalogue$
 - We have $\subseteq$ rather than $=$.
 - Division **is like** the inverse of Cartesian product, **rather than is** the inverse.

Checking the schema:

- Let the schema of *catalogue* be $c = (c_name, p_name)$.
- Let the schema of *programme* be $p = (p_name)$.
- $schema(catalogue \div programme) = (c_name) = c - p$

Division Definition and General Properties

Given the tables R , S on the schemas r and s respectively. The division $R \div S$ has these properties:

- The result for $R \div S$ is a table on the schema $r - s$.
- The tuple t is in $R \div S$ if and only if for **every tuple** t' in S , t **concatenates** t' is a tuple in R .
- In other words, if $Q = R \div S$, then $Q \times S \subseteq R$.
- Note in the previous example, $R = \text{catalogue}$ and $S = \text{programme}$.

Correlated Subquery

- Correlated subquery is a way to implement division in MySQL.
- Recall the schema:
Catalog: $c = (c_name, p_name)$
Programme: $p = (p_name)$
- Typically in database tables, there are other columns that are irrelevant to our query; we can ignore those columns.

Correlated Subquery

- For the query, “Retrieve (all) **course name** that is/are taught by **all** programmes”, the solution is $catalogue \div programme$.
- Suppose we are allowed to use a programming language for this task. Then,

- For each tuple t in *catalogue*
- Select all tuples t' in *catalogue* such that $t'.c_name = t.c_name$
- Put those $t'.p_name$ in *temp*
- If *temp* covers all *p_name* in *programme*
- $t.c_name$ is a tuple in the result

- temp* covers all *p_name* is equivalent to $programme \subseteq temp$,
- which is further equivalent to $programme - temp = \emptyset$.
- Then, the code can be...

```
for each tuple  $t$  in catalogue
   $temp \leftarrow \{\}$ 
  for each tuple  $t'$  in catalogue
    if  $t.c\_name = t'.c\_name$ 
       $temp \leftarrow temp \cup \{t'.p\_name\}$ 
    end if
  end for
  if  $programme - temp = \emptyset$  then
     $t$  is in the result
  end if
end for
```

- We just need to implement it in SQL.

Correlated Subquery Implementation of Division

```
for each tuple t in catalogue
  temp ← {}
  for each tuple t' in catalogue
    if t.c_name = t'.c_name
      temp ← temp ∪ {t'.p_name}
    end if
  end for
  if programme - temp = ∅ then
    t is in the result
  end if
end for
```

Dividend
↓
catalogue
÷
programme
↑
Divisor

```
SELECT C1.c_name
FROM catalogue AS C1
WHERE NOT EXISTS (
```

```
(SELECT p_name
FROM programme)
```

EXCEPT

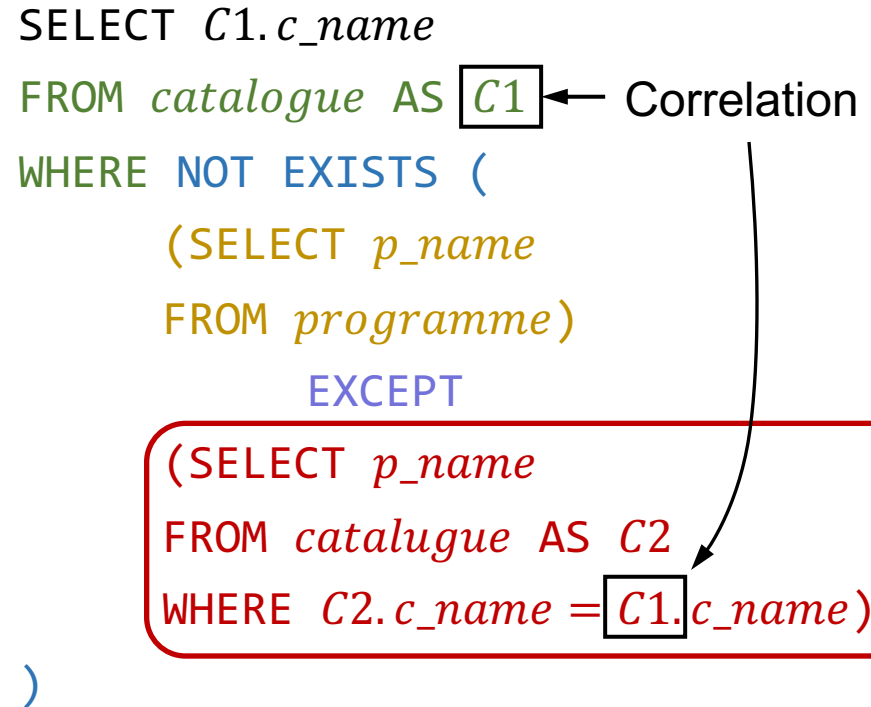
```
(SELECT p_name
FROM catalogue AS C2
WHERE C2.c_name = C1.c_name)
)
```

- Everything from Line 3 to 10 (in NOT EXISTS) is a logical test.
- It is done on every tuple in *C1* on Line 2 .
- To test *programme* - *temp* = ∅, NOT EXISTS is used. It returns TRUE if it is followed by an empty set (in the blue parenthesis (...)).
- To calculate *programme* - *temp*, EXCEPT is used for set minus.

Correlated Subquery Implementation of Division

- This is called a **correlated subquery** because a **correlated name** (*C1* in this example) appears in the outer query, which can be used in the **subquery** (in red).
- The scoping rule is similar to that in C and Java.
 - Treat the box around the subquery like an one-way glass.
 - Inner query can “see” the variables in the outer query, but outer query cannot “see” the variable in the inner query.

```
SELECT C1.c_name
FROM catalogue AS C1 ← Correlation
WHERE NOT EXISTS (
    (SELECT p_name
     FROM programme)
    EXCEPT
    (SELECT p_name
     FROM catalogue AS C2
     WHERE C2.c_name = C1.c_name)
)
```



Correlated Subquery Implementation of Division

- The subquery in the **WHERE** clause will be executed for every tuple from the main query. Thus, correlated subquery is **NOT efficient**.
- (NOT) EXISTS can do many other tasks besides division. Don't be limited.
- A set minus can also be implemented by **NOT IN**.
- **SELECT *** because **NOT EXISTS** only checks whether the set is empty. The tuples in the set are not important.

```
SELECT C1.c_name
FROM catalogue AS C1
WHERE NOT EXISTS (
    SELECT *
    FROM programme
    WHERE p_name NOT IN(
        SELECT C2.p_name
        FROM catalogue AS C2
        WHERE C2.p_name = C1.p_name
    )
)
```

Example

- In the “sakila” database, find the id of the customers who have rented some films from all staffs.

Exercises

Using the UIC database, write a MySQL query to find the following.

1. Which customer(s) has rented some films from all stores?
2. Which customer(s) has rented at least one film in every category?
3. Which customer(s) has rented at least one film in each rating? Assume that every rating has at least one film.

(Hint: use `SELECT DISTINCT` to remove redundant rating)

Save your queries in a txt file. Rename it as “COMP3013 Lab10 ####.txt”, where “####” is your student ID. And submit it on iSpace. The DDL is 24 hours after the lab.

End of Lab 10